Plan — Phase 2: reverse-mode (adjoint) gradients for rotor chains

`hymeko_clifford::autodiff`

Aiko Seto

Created-at: 2026-08-16 20:55 CEST

ETA

≈50 min: 20 min module + contracts, 15 min tests (adjoint vs finite difference vs closed form, operation counts), 10 min benchmark + measurement against the recorded $O(k^2)$ baseline, 5 min report. Basis: the mathematics is a two-sweep recurrence with a known closed form; the correctness harness (`tests/scenario_derivative.rs`) and the cost baseline (`clifford/derivative/*`) both already exist, measured 2026-08-16.

1 Scope

`hymeko_clifford` is documented as the "Clifford-algebra autograd backend for G-SPHF"; Phase 1 shipped no gradients. Measured consequence (`reports/2026-08-16-nagare-cpu-memory-benchmark.md` §6h): a central-difference gradient over a k-angle rotor chain costs $2k$ forward evaluations, i.e. $O(k^2)$ — 63.9× the forward pass at $k = 32$.

This plan adds the reverse sweep, which is $O(k)$.

Not in scope: a general autograd tape over arbitrary multivector expressions. The rotor chain is the shape that is measured, benchmarked, and used; a tape is a larger decision and should follow evidence from this one.

2 Mathematics

With $x_0 = v$, $x_i = R_i x_{i-1} \tilde{R}_i$, $w = x_k$, and rotors $R_i = \exp(-\frac{\theta_i}{2} B_i)$ about unit axes ω_i:

$$\frac{\partial x_i}{\partial \theta_i} = \omega_i \times x_i, \qquad \frac{\partial x_i}{\partial x_{i-1}} = R_i(\cdot)\tilde{R}_i.$$

Seeding the adjoint with $\bar{a}_k = \partial L / \partial w$, the reverse recurrence is

$$\frac{\partial L}{\partial \theta_i} = \bar{a}_i \cdot (\omega_i \times x_i), \qquad \bar{a}_{i-1} = \tilde{R}_i \, \bar{a}_i \, R_i,$$

and $\bar{a}_0 = \partial L / \partial v$ falls out for free. One forward sweep storing $x_0 \dots x_k$, one backward sweep: $2k$ rotor applications and k cross products, $O(k)$ **in both time and memory**, against $2k^2$ rotor applications for the central difference (already asserted by counter in `gradient_of_a_chain_costs_two_evaluations_`

3 Affected files

- `src/autodiff.rs` — new module: `RotorChain`, `Adjoint`, `ChainError`.

- `src/lib.rs` — declare and re-export; the Phase-1 doc comment updated so it no longer claims gradients are absent.
- `tests/scenario_adjoint.rs` — new scenario tests.
- `../../nagare/benches/clifford_algebra_bench.rs` — adjoint arm beside the existing finite-difference sweep, same shapes.
- Mirror every source change to `nagare/vendor/hymeko_clifford/`, which is byte-identical today and must stay so.

4 CORE.YAML items touched

None — `CORE.YAML` does not exist in this workspace (flagged 2026-08-16). **No dependency added**: the module is `std`-only, like the rest of the crate.

5 Interface

```
pub struct RotorChain { /* axes: Vec<[f64; 3]> */ }
pub struct Adjoint { pub value: [f64; 3],
                     pub grad_angles: Vec<f64>,
                     pub grad_input: [f64; 3] }
pub enum ChainError { NonUnitAxis { index, norm }, AngleCount { expected, got } }

impl RotorChain {
    pub fn new(axes: Vec<[f64; 3]>) -> Result<Self, ChainError>;
    pub fn apply(&self, angles: &[f64], v: [f64; 3]) -> Result<[f64; 3],
        ChainError>;
    pub fn adjoint(&self, angles: &[f64], v: [f64; 3],
                   grad_out: [f64; 3]) -> Result<Adjoint, ChainError>;
    pub fn finite_difference_gradient(&self, angles: &[f64], v: [f64; 3],
                                      grad_out: [f64; 3], h: f64)
        -> Result<Vec<f64>, ChainError>;
}
```

`Result` rather than panics: axis normalisation and angle-count are caller errors, and §6.4 forbids swallowing them. The finite difference ships *in the library* deliberately — it is the reference a caller uses to gradient-check its own objective, and keeping it beside the adjoint means the two cannot drift apart unnoticed.

6 Test strategy

Unit adjoint vs the closed form $\omega \times w$ at $k = 1$; error type surfaces a non-unit axis and an angle-count mismatch.

Scenario adjoint vs central difference over $k \in \{1, 2, 4, 8, 16\}$, random axes and seeds; `grad_input` checked against a finite difference in v; gradient of a closed loop is zero where the loop is the identity.

Cost operation counters assert $2k$ rotor applications for the adjoint against $2k^2$ for the difference — the $O(k)$ claim machine-checked, not left to the clock.

Regression the adjoint did not exist, so every test is new; the finite-difference reference is the oracle.

7 Performance budget

Metric ($k = 32$, measured 2026-08-16)	Finite difference	Target (adjoint)
gradient / forward ratio	63.9×	$\leq 4\times$
absolute gradient cost	51.42 µs	≤ 4 µs
growth per doubling of k	$\approx 4\times$	$\approx 2\times$

Targets follow from the operation counts, not from optimism: the adjoint does $2k$ rotor applications where the difference does $2k^2$. If the measurement lands outside, the implementation is wrong or the constant is dominated by allocation — both are findings, not excuses.

8 Rollback

Purely additive: a new module, new tests, a new benchmark arm. Deleting `src/autodiff.rs` and its `pub mod` line restores the crate exactly. No existing signature changes, no data or fixture is mutated.

9 Risk anticipation

- **Worst-case input**: long chains (k up to 32 benchmarked, no upper bound in the API), angles near π where a Cayley parameterisation would diverge — avoided by parameterising the rotor directly as $\cos(\theta/2) + \sin(\theta/2)B$ rather than through `cayley_to_unit_quat`.
- **Precision**: the reference finite difference in `f64` is second-order accurate (verified: halving h cuts error 4.00×). The pre-existing benchmark differences the `f32` quaternion path with $h = 10^{-5}$, which sits near the cancellation limit for `f32`; that is a flaw in the *baseline benchmark*, recorded in the report, and the reason this module and its comparison are `f64` throughout.
- **Contract preservation**: the crate is `#![forbid(unsafe_code)]` and `#![warn(missing_docs)]`; the new module inherits both, so every public item carries preconditions and postconditions.
- **Duplication**: two copies of the crate exist. Both are patched and re-checked with `diff -rq`, as for the `GradRequest` fix.

10 Known deviation

`plan.pdf` cannot be produced: no TeX toolchain on this host, and installing one is a §1 dependency change. Per §11 this is a halt condition; it is recorded and reported rather than skipped. The `.tex`, `.tikz` and `.mmd` sources are complete.